

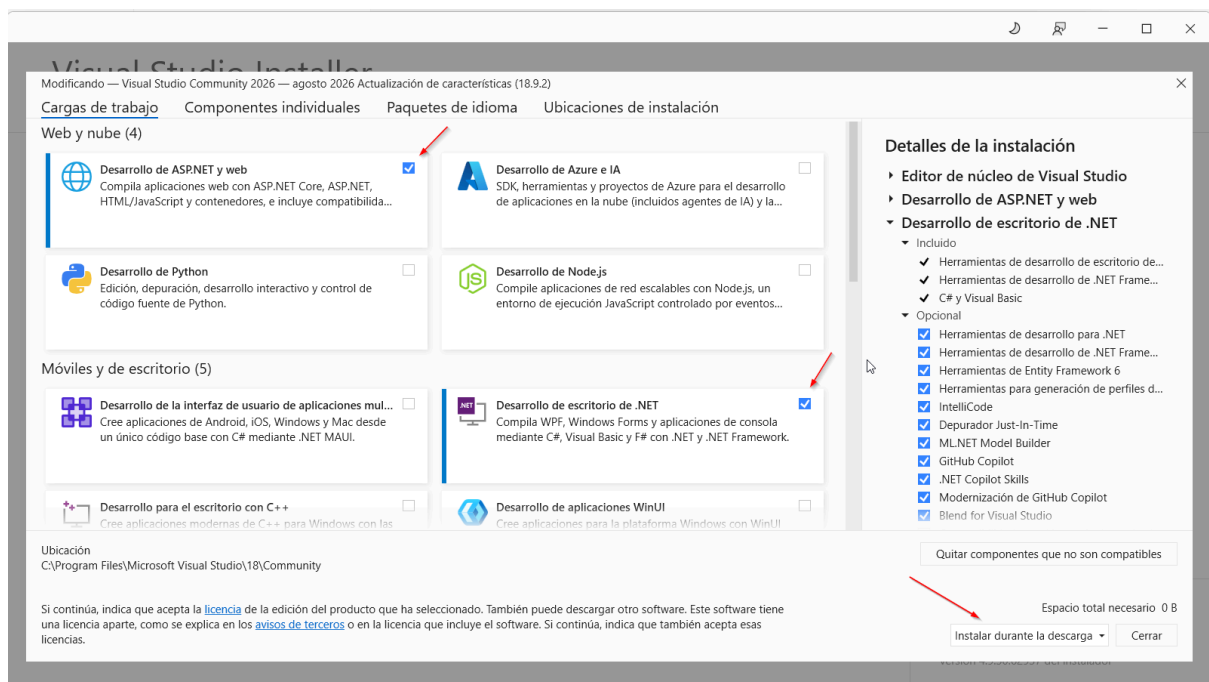
1. Entorno de desarrollo

Opción A — Windows: Visual Studio Community

Para los alumnos de Windows recomiendo **Visual Studio Community** porque les da un entorno completo para aprender C#, depurar y trabajar con proyectos .NET.

Durante la instalación deben seleccionar la carga de trabajo:

Desarrollo de escritorio con .NET / .NET desktop development



Microsoft indica específicamente que esta carga de trabajo permite desarrollar aplicaciones de consola, Windows Forms y WPF con C# y .NET.

[Descargar Visual Studio Community](#)

Pasos para los alumnos

1. Descargar Visual Studio Community

Entrar al sitio oficial y descargar **Community**.

2. Ejecutar el instalador

Al abrirse el instalador:

- seleccionar **.NET desktop development**
- instalar

No necesitan instalar todas las demás cargas de trabajo para comenzar con POO.

3. Abrir Visual Studio

Seleccionar:

Create a new project

Buscar:

Console App

Seleccionar:

C#

y crear el proyecto.

4. Probar C#

El primer programa puede ser:

```
None  
Console.WriteLine("Hola Mundo");
```

Ejecutarlo con:

Ctrl + F5

o desde el botón de ejecución.

2. Opción B — macOS: Visual Studio Code

Acá haría una aclaración importante para tus alumnos:

Visual Studio y Visual Studio Code no son lo mismo.

Para Mac, Microsoft ya no ofrece Visual Studio para macOS, por lo que la alternativa práctica es:

- Visual Studio Code
- .NET SDK
- extensión C#
- C# Dev Kit

El **SDK** es lo que realmente permite crear, compilar y ejecutar las aplicaciones .NET; puede utilizarse desde Visual Studio Code u otro editor.

Paso 1 — Instalar .NET SDK

[Descargar .NET SDK](#)

Actualmente hay varias versiones soportadas de .NET. Microsoft clasifica .NET 10 como LTS activo y .NET 8 como LTS en mantenimiento.

Instalar .NET 8 y trabajar todos con esa versión.

Después de instalarlo, abrir una terminal (CMD en windows por ejemplo):

```
None  
dotnet --version
```

Debería aparecer la versión instalada.

Paso 2 — Instalar Visual Studio Code

[Descargar Visual Studio Code](#)

Instalar normalmente.

Paso 3 — Instalar C# Dev Kit

Abrir VS Code.

Ir a:

Extensions

Buscar:

C# Dev Kit

e instalar la extensión oficial de Microsoft.

También conviene instalar/confirmar la extensión:

C#

Paso 4 — Crear el proyecto

Desde la terminal:

```
None  
dotnet new console -n P00Clase01
```

Entrar al proyecto:

```
None  
cd P00Clase01
```

Ejecutar:

```
None  
dotnet run
```

Deberían obtener:

```
None  
Hello, World!
```

3. Actividad inicial

Antes de empezar con los pilares.

Laboratorio 0 – Mi primer proyecto C#

Objetivo

Comprobar que el entorno está correctamente instalado.

Crear una aplicación de consola que muestre:

```
None  
=====
```

```
DESARROLLO BACK END
```

```
=====
```

Alumno: Juan Perez
Materia: Programación Orientada a Objetos
Lenguaje: C#

Esto te permite detectar **problemas de instalación antes de entrar en POO**.

4. Ahora sí: POO

Requerimiento:

Una universidad necesita un sistema para administrar alumnos. Cada alumno posee nombre, DNI y legajo. El sistema debe permitir registrar alumnos y mostrar sus datos.

¿Qué sustantivos encontramos?

None
universidad
sistema
alumnos
nombre
DNI
legajo
datos

¿Cuáles podrían ser clases?

None
Alumno

¿Cuáles podrían ser atributos?

None
nombre
dni
legajo

¿Qué verbos encontramos?

None

```
administrar  
registrar  
mostrar
```

Y entonces introducir:

None

```
public class Alumno  
{  
    public string Nombre { get; set; }  
    public string Dni { get; set; }  
    public int Legajo { get; set; }  
  
    public void MostrarDatos()  
    {  
        Console.WriteLine($"Alumno: {Nombre}");  
        Console.WriteLine($"DNI: {Dni}");  
        Console.WriteLine($"Legajo: {Legajo}");  
    }  
}
```

Y crear el objeto:

None

```
Alumno alumno = new Alumno();  
  
alumno.Nombre = "Juan";  
alumno.Dni = "30123456";  
alumno.Legajo = 1001;  
  
alumno.MostrarDatos();
```

Ahí ya tenés:

Clase → objeto → atributos → método.

5. Ejercicios progresivos para los cuatro pilares

Ejercicio 1 — Clase y objeto

Requerimiento

Una empresa necesita administrar sus empleados.

Cada empleado posee:

- nombre
- DNI
- salario

El sistema debe permitir mostrar los datos del empleado.

El alumno debe identificar

None

Clase:

Atributos:

Métodos:

Y posteriormente implementarlo en C#.

Objetivo

Introducir:

- clase
- objeto
- atributo
- método
- constructor

Ejercicio 2 — Encapsulamiento

Requerimiento

Un banco necesita administrar cuentas bancarias.

Cada cuenta tiene:

- número
- titular
- saldo

El saldo **no puede modificarse directamente** desde fuera de la clase.

La cuenta debe permitir:

- depositar dinero
- retirar dinero
- consultar saldo

No se puede retirar más dinero del disponible.

Pregunta clave

¿Qué está mal en esto?

None

```
cuenta.Saldo = -50000;
```

Los alumnos deberían llegar a que el objeto debe **proteger su estado interno**.

Una implementación posible:

None

```
public class Cuenta
{
    public string Numero { get; set; }
    public string Titular { get; set; }

    private decimal saldo;

    public void Depositar(decimal monto)
    {
        if (monto > 0)
            saldo += monto;
    }

    public bool Retirar(decimal monto)
```

```
{
    if (monto <= 0 || monto > saldo)
        return false;

    saldo -= monto;
    return true;
}

public decimal ConsultarSaldo()
{
    return saldo;
}
}
```

Concepto

Encapsulamiento = proteger el estado interno y controlar cómo se modifica.

Ejercicio 3 — Abstracción

Sistema de vehículos

Una empresa posee diferentes vehículos.

Todos los vehículos tienen:

- patente
- marca
- modelo

Todos deben poder:

Arrancar()

Pero la forma de arrancar puede ser diferente dependiendo del vehículo.

Crear una clase abstracta:

None

Vehiculo

y luego:

None

Auto

Moto

Camion

Preguntas

1. ¿Qué características comparten?
2. ¿Qué comportamiento comparten?
3. ¿Qué comportamiento podría ser diferente?
4. ¿Qué debería quedar en Vehiculo?

Acá introducís:

None

```
abstract class Vehiculo
```

y métodos abstractos.

Ejercicio 4 — Herencia

A partir del ejercicio anterior:

None

```
Vehiculo
```

```
|
```

```
+----- Auto
```

```
|
```

```
+----- Moto
```

```
|
```

```
+----- Camion
```

Los alumnos deberán implementar la relación de herencia.

Por ejemplo:

```
None
public class Auto : Vehiculo
{
}
```

Y agregar características específicas:

```
None
Auto:
cantidadPuertas

Moto:
cilindrada

Camion:
capacidadCarga
```

Objetivo

Comprender:

es-un

Un Auto **es un** Vehículo.

Una Moto **es un** Vehículo.

Ejercicio 5 — Polimorfismo

Continuamos exactamente con los vehículos.

Cada vehículo debe implementar:

```
None
Acelerar()
```

pero cada uno lo hace de una manera diferente.

Por ejemplo:

None

```
Vehiculo auto = new Auto();
Vehiculo moto = new Moto();
Vehiculo camion = new Camion();

auto.Acelerar();
moto.Acelerar();
camion.Acelerar();
```

La llamada es la misma:

None

```
Acelerar()
```

pero el comportamiento depende del objeto.

Este es el momento ideal para explicar:

Una misma interfaz (clase abstracta) puede representar comportamientos diferentes según el objeto concreto.

Ejercicio 6 — Sistema de empleados

Este ejercicio combina los pilares.

Una empresa tiene empleados:

- Empleado Administrativo
- Empleado Vendedor
- Empleado Gerente

Todos tienen:

- nombre
- DNI
- salario básico

Pero el cálculo del salario es diferente.

Administrativo

None

```
salario = salario básico
```

Vendedor

None

```
salario = salario básico + comisión
```

Gerente

None

```
salario = salario básico + bono
```

Crear:

None

```
Empleado  
|  
+--- Administrativo  
+--- Vendedor  
+--- Gerente
```

Y utilizar:

None

```
List<Empleado>
```

para almacenar empleados de diferentes tipos.

Después:

None

```
foreach (Empleado empleado in empleados)  
{  
    Console.WriteLine(empleado.CalcularSalario());  
}
```

Ejercicio 7 — Composición

Después de enseñar herencia, yo introduciría deliberadamente un problema donde **NO conviene usar herencia**.

Requerimiento

Un automóvil tiene:

- motor
- cuatro ruedas
- batería

El motor puede:

- arrancar
- detenerse

El automóvil puede:

- arrancar
- detenerse

Pregunta

¿Deberíamos hacer?

None

Auto hereda de Motor

✗ No.

Porque:

un Auto **no es un Motor**.

En cambio:

un Auto **tiene un Motor**.

Entonces:

None

Auto

└─ Motor

└─ Bateria

Esto te permite introducir **composición**.

Ejercicio 8 — Integrador de POO

Finalmente les daría un requerimiento más parecido a un parcial.

Sistema de biblioteca

Una biblioteca necesita un sistema para administrar libros y socios.

Cada libro posee:

- ISBN
- título
- autor
- año de publicación

Cada socio posee:

- DNI
- nombre
- número de socio

Un socio puede solicitar el préstamo de un libro.

Un libro no puede prestarse si ya está prestado.

El socio puede devolver el libro.

El sistema debe registrar los préstamos realizados.

Los alumnos deben poder hacer

1. Análisis

None

Sustantivos

↓

Clases

Características

↓

Atributos

Verbos

↓

Métodos

2. Diagrama

None

Socio

Libro

Prestamo

3. Código C#

Aplicar:

- clases
- objetos
- encapsulamiento
- abstracción donde corresponda
- composición/asociación
- métodos
- validaciones

4. Programa principal

Crear varios objetos:

None

2 socios

5 libros

y simular:

None

registrar socio

registrar libro

prestar libro

intentar prestar libro ya prestado

devolver libro

volver a prestar